SkillVerify Project Report (Detailed, Simple Language)

1. Problem Definition

Hiring teams often depend on resumes, but resumes may not fully prove real technical ability. A candidate can list many skills, but employers still need evidence.

This project solves that problem by checking whether resume claims are supported by real GitHub work.

The system takes a resume and/or GitHub username, analyzes repositories, compares claimed skills with observed technologies, and generates an evidence-based report.

---

2. Significance and Objectives

2.1 Significance

This project is important because:

1. It reduces bias from purely manual resume screening.
2. It provides objective, repeatable signals from real code activity.
3. It helps recruiters save time by ranking candidates with data.
4. It gives candidates feedback on strengths and improvement areas.
5. It supports both single and bulk analysis for practical hiring workflows.

2.2 Objectives

The main objectives are:

1. Parse resume files (PDF/DOCX) and extract candidate details.
2. Extract or accept GitHub username directly.
3. Fetch and analyze GitHub profile and repositories.
4. Detect technologies and frameworks used in real projects.
5. Match resume skills with GitHub evidence.
6. Measure repository quality and contribution behavior.
7. Compute an explainable final score.
8. Produce readable dashboard and downloadable report artifacts.

---

3. System Requirements

3.1 System Requirements (Functional and Non-Functional)

3.1.1 Functional Requirements

1. User must be able to upload a single resume (PDF/DOCX).
2. User must be able to upload a bulk ZIP of resumes.
3. User must be able to enter a GitHub username directly without uploading resume.
4. System must extract candidate info (name, email, phone, GitHub link if present).

5. System must fetch GitHub profile and repositories.
6. System must detect skills/technologies from repositories.
7. System must compare resume skills vs detected GitHub skills.
8. System must calculate repository quality metrics and grades.
9. System must analyze contribution patterns (commits, activity, streaks).
10. System must generate final score and recommendation.
11. System must show detailed dashboard visualization.
12. System must allow report export (JSON and PDF).
13. System must support robust error messages for missing data and invalid files.

3.1.2 Non-Functional Requirements

1. Performance:

- Single resume analysis should finish in acceptable time (depends on GitHub API response and repo count).
- Bulk processing should isolate failures per file and continue remaining files.

2. Reliability:

- Parser should use fallback methods for PDF extraction.
- Repo-level errors should not crash whole analysis.

3. Scalability:

- System should process multiple candidates using independent per-candidate pipeline.
- Sampling and caps should control API usage and compute load.

4. Security:

- Validate file types and file size.
- Use environment variables for secrets (GITHUB_TOKEN).
- Sanitize file names and clean temporary files.

5. Usability:

- Provide clear status/errors in UI.
- Show interpretable component-level score breakdown.

6. Maintainability:

- Modular service design in backend.
- Clear separation of parsing, fetching, analysis, scoring, and reporting.

7. Portability:

- Should run on local development machine with Python + Node setup.

---

4. Tech Stack and Important Libraries with Justification

4.1 Frontend Stack

1. React 18

- Why: Component-based UI, reusable structure, strong ecosystem.

2. Vite

- Why: Fast dev startup and modern bundling.

3. Axios

- Why: Simple and reliable HTTP calls to backend APIs.

4. Recharts

- Why: Good chart components for score and activity visualization.

5. react-table

- Why: Sorting and tabular support for bulk candidate dashboards.

6. lucide-react

- Why: Clean icon set for professional dashboard presentation.

4.2 Backend Stack

1. Flask

- Why: Lightweight and clear for REST API orchestration.

2. Flask-CORS

- Why: Enables frontend-backend communication across local ports.

3. python-dotenv

- Why: Loads API secrets and environment config cleanly.

4. requests

- Why: Stable HTTP client for GitHub API integration.

5. pdfplumber and PyPDF2

- Why: PDF extraction with fallback for reliability.

6. python-docx

- Why: DOCX resume parsing support.

7. PyMuPDF

- Why: Better extraction of embedded links, especially GitHub URLs in PDFs.

8. ReportLab and Playwright

- Why: ReportLab for structured fallback PDF, Playwright for high-fidelity HTML-to-PDF export.

9. Optional quality/analysis libraries

- textstat, radon, esprima, javalang, rapidfuzz
- Why: Improve readability analysis, code-health quality metrics, and fuzzy matching.

---

5. Module-Wise Algorithms and Pseudocode

Below is simplified pseudocode for each important module.

5.1 backend/app.py (Pipeline Orchestrator and API)

Purpose:

- Accept user input, run complete analysis pipeline, return report.

Pseudocode:

1. Receive request.
2. Validate input mode:

- single resume
- bulk zip
- github username only

3. For each candidate:

- parse resume or build pseudo-resume
- fetch github data
- analyze tech stack
- if resume has skills: perform skill matching
- else: mark skill matching skipped
- analyze repository quality
- analyze contributions
- compute scores
- generate report
- optionally save deep scoring html

4. Return JSON response.

5.2 resume_parser.py

Purpose:

- Extract structured candidate information from resume.

Pseudocode:

1. Detect file type.
2. If PDF:

- try pdfplumber extraction
- if fail, try PyPDF2

3. If DOCX:

- use python-docx paragraphs

4. Extract name/email/phone by regex and heuristics.
5. Extract github URL by pattern.
6. Extract skills using keyword list and normalization map.
7. Return parsed candidate dictionary.

5.3 pdf_link_extractor.py

Purpose:

- Improve GitHub profile detection from PDFs.

Pseudocode:

1. Open PDF with PyMuPDF.
2. Read links and text.
3. Filter GitHub-like URLs.
4. Remove non-profile patterns.
5. Choose best username candidate.
6. Return success/failure with details.

5.4 github_service.py

Purpose:

- Central GitHub API integration.

Pseudocode:

1. Normalize username from URL or raw input.
2. Fetch user profile.
3. Fetch repositories with pagination.
4. For each repo (as needed):

- fetch contents/languages/commits/contributors

5. Aggregate language and contribution stats.
6. Handle API errors (404, rate limit, network).
7. Return consolidated GitHub data.

5.5 tech_stack_analyzer.py

Purpose:

- Detect technologies used in repositories.

Pseudocode:

1. Select top non-fork repos by stars and recency.
2. For each selected repo:

- inspect root files and directory structure
- inspect manifests (package.json, requirements.txt, etc.)
- inspect source patterns/import clues

3. For each detected technology:

- assign type, confidence, evidence
- track usage repositories

4. Aggregate all detections across repos.
5. Return detected_skills + repo_details.

5.6 skill_matcher.py

Purpose:

- Compare resume-declared skills with GitHub-detected skills.

Pseudocode:

1. Normalize resume skills.
2. Normalize github detected skills.
3. For each resume skill:

- try exact canonical match
- else try fuzzy match
- if found: add matched_skills
- if not found: add missing_skills

4. Identify extra github skills not listed in resume.
5. Compute match percentage and authenticity score.
6. Return matching result object.

5.7 code_quality_analyzer.py

Purpose:

- Grade repository quality using weighted evidence.

Pseudocode:

1. Select top non-fork repos for quality analysis.
2. For each repo:

- fetch root contents
- compute documentation score
- compute code organization score

- compute commit quality score
- optionally compute code health score
- combine category scores with weights
- calibrate score and assign grade

3. Aggregate portfolio metrics and suggestions.
4. Return overall quality report.

5.8 contribution_analyzer.py

Purpose:

- Analyze activity and ownership behavior from commits.

Pseudocode:

1. Separate owned repos and fork repos.
2. For priority repos:

- fetch commits
- count user commits and total commits
- compute ownership percentage
- track last contribution and activity

3. Build monthly and weekday activity summaries.
4. Compute streak and consistency indicators.
5. Return contribution summary and top repositories.

5.9 scoring_engine.py

Purpose:

- Convert analysis outputs into final score.

Pseudocode:

1. Read component values:

- skill authenticity
- code quality
- commit activity
- tech stack
- profile signal

2. If skill matching skipped:

- remove skill component from active weights
- normalize remaining weights

3. Compute weighted overall score.
4. Map overall score to rating band.
5. Return overall + breakdown + meta.

5.10 report_generator.py

Purpose:

- Build structured final report and export-friendly outputs.

Pseudocode:

1. Collect all component outputs.
2. Build report sections:

- candidate summary
- github profile summary
- technical analysis
- skill matching report
- code quality report
- contribution report
- scoring and recommendation

3. For PDF download:

- render HTML with Playwright
- fallback to ReportLab when needed

4. Return report.

5.11 Frontend Upload Module (UploadSection)

Purpose:

- Input validation and file mode handling.

Pseudocode:

1. User chooses mode: single or bulk.
2. Validate selected file type and size.
3. Build FormData request.
4. Call upload endpoint.
5. Show progress and result or error.

5.12 Frontend Dashboard Modules

Purpose:

- Present analysis in readable and actionable format.

Pseudocode:

1. Receive report JSON.
2. Render overview cards and tabs.
3. Render charts/tables for skills, quality, contributions, profile.
4. If skill matching skipped:

- hide skill-specific and overall sections (as configured)
- show individual component metrics

5. Build downloadable report template.

---

6. Experimental Setup and Test Criteria

6.1 Experimental Setup

Environment baseline:

1. OS: Windows (development machine)
2. Backend runtime: Python 3.x virtual environment
3. Frontend runtime: Node.js + Vite
4. Backend API server: Flask on localhost
5. Frontend app: Vite dev server on localhost
6. External dependency: GitHub REST API
7. Optional token: GITHUB_TOKEN for stable rate limits

Dataset design:

1. Single-candidate resumes with clear GitHub links.
2. Resumes with missing/invalid GitHub links.
3. Username-only input mode.
4. Bulk zip with mixed valid and invalid files.
5. Candidates with high, medium, and low repository activity.

6.2 Test Criteria

Functional criteria:

1. Correct parsing for PDF and DOCX.
2. Correct GitHub extraction from resume.
3. Correct API response format and status codes.
4. Proper score breakdown generation.
5. Proper handling of no-skill-matching scenario.
6. Successful report download.

Quality criteria:

1. Response time acceptable for normal single analysis.
2. No crash on one bad file in bulk mode.
3. Clear and actionable error messages.
4. Consistent score ranges (0 to 100).

---

7. Testing Process

The testing process can be documented in five layers.

7.1 Unit-Level Validation

1. Validate parser functions for contact and skills extraction.
2. Validate matching logic for exact/fuzzy skill mapping.
3. Validate score formulas with fixed known inputs.

7.2 Service-Level Validation

1. Test GitHub fetch service with valid and invalid usernames.
2. Test technology analyzer with repositories of different languages.
3. Test code quality analyzer with different repo structures.

7.3 API-Level Validation

1. Test upload endpoint with:

- valid PDF/DOCX
- unsupported file type
- missing file
- username-only request

2. Test bulk endpoint with mixed ZIP contents.
3. Test download-report endpoint for valid HTML payload.

7.4 End-to-End Validation

1. Upload resume from frontend.
2. Verify dashboard sections and metrics.
3. Download generated PDF and inspect content.
4. Repeat for username-only mode.

7.5 Regression Checks

1. Ensure new changes do not break older modes.
2. Verify no-skill mode still shows component-level insights.
3. Verify skipped skill matching does not produce misleading totals when hidden.

---

8. Challenges Faced and Solutions

8.1 Challenge: Resume text extraction inconsistency

Problem:

- Different PDF formats produce different text quality.

Solution:

- Use multi-parser strategy (pdfplumber primary, PyPDF2 fallback).
- Add dedicated link extraction from PDF using PyMuPDF.

8.2 Challenge: GitHub rate limits

Problem:

- Unauthenticated calls can hit API limits.

Solution:

- Support GITHUB_TOKEN in environment.
- Keep repository sampling caps in analyzers.

8.3 Challenge: Repository diversity

Problem:

- Repositories vary by language and project maturity.

Solution:

- Use weighted multi-factor scoring.
- Use per-repo calibration and confidence-based evidence.

8.4 Challenge: Resume skill mismatch edge cases

Problem:

- User may provide username only (no resume skills).

Solution:

- Skip skill matching safely.
- Compute component metrics from remaining modules.
- Hide overall score where required to avoid confusion.

8.5 Challenge: Explainability of scores

Problem:

- Black-box scoring is hard to trust.

Solution:

- Expose score breakdown and report-level details.
- Include explicit formulas and category metrics.

---

9. Conclusion

This system provides a practical, evidence-driven hiring analysis pipeline by connecting resume claims to actual GitHub activity.

It combines parsing, repository intelligence, quality heuristics, contribution analysis, and explainable scoring into one workflow.

The architecture is modular and suitable for further extension in real recruitment environments.

---

10. Recommendations

1. Add caching layer for GitHub API responses to improve speed and reduce API calls.
2. Add asynchronous job queue for large bulk uploads.
3. Add persistent storage for historical reports and comparison over time.
4. Add interviewer-focused role templates (backend, frontend, data science, DevOps).
5. Add stronger evaluation metrics with benchmark datasets.
6. Add automated test suite with CI for parser/scoring regressions.
7. Add fairness checks and configurable weighting by role.
8. Add observability logs and dashboards for production deployment.

---

11. Suggested Annex for Academic Report

You can attach these items as appendices:

1. API endpoint definitions and sample payloads.
2. Screenshot set of each dashboard tab.
3. Sample generated JSON report.
4. Sample generated PDF report.
5. Formula table for all scores.
6. Module dependency diagram.
7. Test case matrix with expected outputs.

---

12. Quick Reference Summary

1. Problem: Resume claims need verification.
2. Approach: Compare resume and GitHub evidence using modular analyzers.
3. Output: Structured report, dashboard visualization, exportable PDF.
4. Value: Better, faster, and more explainable technical candidate screening.